PERSONAL • ENGINEERING • VISION

NASIR HAYAT

Personal & Technical Identity Profile

BUILDING SYSTEMS.

SECURING DELIVERY.

EXPLORING INTELLIGENCE.

A practical snapshot of my background,
projects, engineering approach, and direction.

Prepared October 2026 • Based on documented conversations and project context

A clear, honest overview of who I am, what I am building, what I have learned, and where I intend to go.

Profile principle: This document distinguishes completed work from active work, learning, and future ideas. It is a working profile—not a claim that every listed technology is already mastered or every project is production-ready.

THE SHORT VERSION

01 — Executive Snapshot

Nasir Hayat is a Computer Science student in the 7th semester who is deliberately building toward a career in DevOps and DevSecOps, while exploring cloud-native engineering, full-stack software, and AI-powered automation. His learning style is practical: understand the concept, build a working system, test it, inspect failure modes, document the result, and improve the architecture. His strongest evidence is project work—especially a Notes web application used as a local DevSecOps lab with automated CI checks, container images, Kubernetes deployment, GitOps delivery, and application security testing.

A useful professional description is: **DevOps / DevSecOps engineer in training, with hands-on cloud-native project experience and an emerging focus on AI automation and full-stack product building.** This wording reflects demonstrated direction without overstating seniority or claiming mastery of every adjacent discipline.

AREA	CURRENT PROFILE
Name	Nasir Hayat
Education	Computer Science student; 7th semester as previously shared
Primary direction	DevOps, DevSecOps, automation, containers, Kubernetes, CI/CD and GitOps
Adjacent interests	AI automation / agentic systems, full-stack applications, browser extensions, product design
Learning approach	Concept-first, hands-on labs, active recall, spaced repetition, troubleshooting, documentation
Long-term ambition	Become a highly capable engineer, develop CTO-level systems thinking, and pursue an international career including Australia

Public-facing links (verify before sharing)

- Portfolio: nasirhayat.me
- GitHub: github.com/nasirhayat028

This PDF intentionally does not include private credentials, tokens, account numbers, or other sensitive account details.

PERSONAL NARRATIVE

02 — About Me

I am a Computer Science student building a practical engineering foundation around how software is developed, tested, secured, deployed, operated, and improved. I am interested in the full lifecycle—not only writing code, but also the infrastructure and automation that make applications repeatable, observable, and safer to release. DevOps and DevSecOps are my main professional direction because they combine systems thinking, Linux, networking, containers, cloud-native platforms, automation, and security. Alongside that, I explore AI automation and product development because I want to understand how useful intelligent features can be designed, integrated, and delivered as real software.

I prefer learning by building real projects rather than collecting disconnected tutorials. My approach is to first understand why a component exists, then verify it manually, automate the repeatable checks, inspect the logs and reports, and improve the system in small, explainable steps. I value discipline, long-term skill development, clear documentation, secure defaults, and evidence over claims. I am still learning, and I treat that as part of the engineering process: define the current state honestly, identify the next gap, and close it through deliberate practice.

My longer-term aim is to become a highly capable engineer who can reason across application code, infrastructure, delivery pipelines, security, and product architecture. I want to grow from implementing individual tools into designing reliable systems and making sound technical decisions. I am also working toward a strong professional portfolio and an international career path, with Australia as a stated future goal.

Personal details known from prior conversations

DETAIL	WHAT IS KNOWN	HOW TO PRESENT IT
Name	Nasir Hayat	Use as profile name
Age	19, as previously shared	Include only if you want age public
Education	CS 7th-semester student, as previously shared	Education / current stage
Career target	Become a world-class DevOps engineer by 25; pursue Australia by 23 was a stated ambition	Present as goals, not guarantees
Personal priorities	Discipline, mastery, smart execution, long-term freedom and family aspirations	Optional; omit from a strictly professional copy if preferred

Sharing note: the age and personal ambitions are included because they were previously shared, but they are optional. Remove them if this PDF is intended for a public portfolio or recruiter rather than a trusted person.

IDENTITY WITHOUT OVERCLAIMING

03 — What I Am Professionally

IDENTITY	WHAT IT MEANS IN MY JOURNEY	EVIDENCE / STATUS
DevOps engineer in training	Automating build, test, package, deployment, and operational workflows so software delivery becomes repeatable.	Hands-on Notes App CI/CD and local Kubernetes/GitOps lab.
DevSecOps practitioner in training	Building security checks into the development and delivery lifecycle instead of treating security as a final step.	Gitleaks, npm audit, SonarQube, Trivy, OWASP ZAP baseline work and response-header hardening in the Notes App project.
Cloud-native learner / builder	Learning how containers, Kubernetes, services, ingress, and GitOps fit together to run an application.	kind cluster, Docker images, Kubernetes namespace/services/Ingress, Argo CD.
AI automation / agentic AI explorer	Designing ideas where AI models or agents assist with multi-step work, comparisons, decisions, or content workflows.	AI MCQ consensus extension and social-media handler agent are design/build directions; do not present all planned capabilities as shipped.
Full-stack product builder	Connecting frontends, APIs, data stores, authentication, deployment and user experience into usable products.	Notes web app and TeamSyncAI project work; completion differs by feature.
Systems thinker / designer	Reasoning about components, boundaries, security, workflows, user journeys, trade-offs, and future extensibility.	Project architecture planning, GitOps/security workflow design, portfolio product strategy.

Skills map — a practical, non-exhaustive snapshot

DOMAIN	TOOLS / CONCEPTS	MATURITY SIGNAL
Linux & CLI	Linux shell, command-line troubleshooting, logs, file paths, environment and networking basics	Active learning and daily practice
Source control & CI	Git, GitHub, GitHub Actions, workflow permissions, automated tests and build stages	Used in project pipeline
Containers	Docker, image tags, Docker Hub, container networking	Used for application images and ZAP container connectivity
Kubernetes & GitOps	kind, namespaces, Pods, Services, Ingress, Argo CD	Local project environment
Security & quality	Gitleaks, npm audit, SonarQube, Trivy, OWASP ZAP, CSP and HTTP security headers	Integrated or tested in the Notes App workflow; scope evolves
Application engineering	React/TypeScript frontend context, Node.js backend, API testing, MongoDB, automated tests	Project experience; depth varies by component
Android & Firebase	Kotlin, Jetpack Compose, Hilt, Firebase Auth/Firestore/Storage	TeamSyncAI project work; some features/testing remain pending
AI & product design	Provider abstraction, multi-model consensus concepts, agent workflows, portfolio interaction and selective 3D	Exploration/design and project planning; avoid implying all productionized

PROJECT HIGHLIGHT 01

04 — Flagship Project: Notes Web App + DevSecOps

This is the clearest end-to-end DevOps/DevSecOps learning environment in the documented work. The Notes web app is used to practice not only application delivery, but also automated verification, image security scanning, Kubernetes deployment, GitOps reconciliation, and Dynamic Application Security Testing (DAST). The environment is intentionally local for now: Docker Hub plus a kind Kubernetes cluster. AWS was explicitly postponed rather than being part of the current delivery target.

Project status

COMPONENT	DOCUMENTED STATE
Source repository	GitHub: nasirhayat028/notes-webapp; main branch in the documented workflow.
CI pipeline	GitHub Actions replaced Jenkins. Frontend/backend jobs run npm ci, tests and build checks; security and quality steps are included.
Test evidence	Frontend Vitest: 1 file / 1 pass in the recorded run. Backend tests: 1 file / 7 passes in the recorded run. These are snapshots, not a claim of exhaustive coverage.
Supply-chain / static checks	Gitleaks secret scan, npm audit high-severity threshold, SonarQube analysis, and Trivy image scanning were incorporated into the workflow.
Container registry	Docker Hub image tags use build-number style tags (example previously observed: build-69). Exact current tag should be checked in the repository/workflow.
Runtime	Local kind cluster named dev-cluster; namespace noteapp; frontend, backend and MongoDB workloads; NGINX Ingress for notes.local.
GitOps	Argo CD application named notes-app in namespace argocd; Argo CD UI host previously configured as argocd.local.
DAST	OWASP ZAP baseline scan run from a Docker container connected to the kind network. A CSP header finding was investigated and enforcing CSP/security headers were added and manually verified.

Pipeline workflow — high-level

STAGE	PURPOSE	EXAMPLE CHECK / OUTPUT
1. Source	Fetch the repository version being validated.	GitHub Actions checkout on push / pull request to main.
2. Install & test	Reproducibly install dependencies and validate behavior.	npm ci; frontend and backend tests.
3. Build	Confirm the app can be packaged for delivery.	Frontend build and backend image build.
4. Secret/dependency/code checks	Catch leaked secrets, known vulnerable packages, and code-quality issues.	Gitleaks, npm audit, SonarQube.
5. Image scan	Inspect built container images for known vulnerabilities.	Trivy scans; preferred readability order is backend build → backend scan → frontend build → frontend scan.
6. Publish & deploy	Make versioned images available and reconcile Kubernetes manifests.	Docker Hub tags; Argo CD / GitOps deployment flow.
7. Runtime verification	Confirm the deployed app works in the target environment.	notes.local, ingress routing, frontend/backend/MongoDB services.
8. DAST	Probe the running web surface for common security issues.	OWASP ZAP baseline report; review findings and remediate.

HOW THE PARTS CONNECT

05 — Notes App: System Architecture

Logical architecture

Developer	Writes code, reviews changes and pushes to GitHub.
↓	
GitHub Actions CI	Runs tests, build steps, secret/dependency/code checks and image vulnerability scans.
↓	
Docker Hub	Stores versioned frontend and backend container images.
↓	
Argo CD	Watches desired Kubernetes configuration and reconciles the cluster toward the declared state.
↓	
kind / dev-cluster	Runs the local Kubernetes workloads in namespace noteapp.
↓	
NGINX Ingress	Accepts browser traffic for notes.local and routes / to frontend and /api/ to backend.
↓	
Frontend	Serves the web interface; browser API calls use relative /api paths.
↓	
Backend API	Handles application API requests and connects to the data layer.
↓	
MongoDB	Stores application data for the backend.
↓	
OWASP ZAP	Runs a baseline scan against the reachable application URL; reports are reviewed separately.

Network and runtime details recorded

- kind cluster: dev-cluster; control-plane node observed as dev-cluster-control-plane.
- Kubernetes namespace: noteapp. Recorded services: backend ClusterIP on 5000, frontend on 80, MongoDB on 27017.
- Ingress host: notes.local; /api/ routes to backend:5000 and / routes to frontend:80.
- ZAP container reachability was addressed by using the kind Docker network and mapping notes.local to the kind node IP for the scan container.
- Security headers in frontend NGINX configuration included X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, and an enforcing Content-Security-Policy.

Security workflow: baseline scan → inspect alert and affected URLs → identify the missing/weak control → implement a narrow fix → verify NGINX config and response headers → retest application behavior and rerun the scan. The recorded CSP issue was “Content Security Policy (CSP) Header Not Set”; the project work moved from report-only CSP to an enforcing policy.

PROJECT PORTFOLIO

06 — Other Product & Engineering Work

TeamSyncAI — Android collaboration app

A Kotlin-based Android collaboration product intended to support organization workspaces and team coordination. The project uses Jetpack Compose for UI, Hilt for dependency injection, and Firebase services for authentication and data. Its architecture uses repository/ViewModel patterns and role-aware flows.

- **Core product areas explored or implemented:** authentication/onboarding, workspaces, role-based Owner/Admin/Manager/Employee access, chat with mentions and reactions, attendance/break tracking, announcements, alerts, profiles/account deletion, and employee-safe team overview.
- **Security and tenancy concerns:** organization isolation, role permissions, invite expiry, and Firebase security rules are part of the design context.
- **Testing snapshot:** a prior hardening pass recorded 19/19 unit tests passing, debug/lint and unsigned release/R8 builds verified, and manual review of login, attendance and tasks reported working. Emulator, live FCM, device testing and production signing were still pending in that snapshot.
- **Status honesty:** not every planned capability was finished. Some functions, email/AI integrations, reviews, reports, push behavior, advanced chat, and other flows were previously identified as stubs or future work. Recheck current repository state before presenting a feature as shipped.

AI MCQ Verification / Consensus Browser Extension

A separate AI product concept aimed at helping users verify multiple-choice answers by comparing outputs from different AI providers, such as OpenAI, Gemini, and Anthropic. The key idea is not to blindly accept one model answer; it is to surface agreement, disagreement, uncertainty, and the need for human review.

- **Intended workflow:** detect or capture a supported MCQ → normalize the question and answer options → query selected AI providers → compare answers and confidence/evidence signals → identify consensus or disagreement → present a result for user review.
- **Architecture discussed:** Chrome Manifest V3 extension, TypeScript/React UI, local extension storage, a backend orchestration layer, schema validation, provider abstraction, and a data store such as SQLite/PostgreSQL if needed.
- **Important safeguards:** model agreement is not proof of correctness; ambiguity should be flagged, evidence should be shown where possible, secrets/API keys should not be exposed in the extension, and privacy/cost/latency should be considered.
- **Status:** architecture and product planning are documented; do not describe every proposed provider, backend feature, or consensus capability as production-ready without fresh verification.

Social Media Handler Agent — AI automation idea

A longer-term idea for an AI-assisted social media workflow across platforms such as Instagram, LinkedIn, TikTok, Facebook and YouTube. The goal is to explore how agents can help with content planning, drafting, repurposing, and operational workflows while preserving human review and platform-compliant behavior.

The concept should be presented as an idea/roadmap unless a working implementation has been separately verified. A responsible version would use official APIs where available, clear permissions, human approval for publishing, and secure handling of credentials.

Portfolio website — nasirhayat.me

The portfolio direction is “Editorial Minimalism”: a professional, distinctive site that communicates real engineering work rather than looking like a generic template. The content should demonstrate DevOps, DevSecOps, automation, full-stack apps, browser extensions, and design awareness. The agreed visual direction uses an ivory/apricot/natural/organic palette, Inter typography, left-aligned editorial presentation, and selective meaningful 3D rather than a fully 3D experience.

- Palette documented: Ivory #F5E6C5, Apricot #D78830, Natural #9F886F, Organic #3F4227.
- Hero concept: “Turning infrastructure into automation”.
- Design intent: premium editorial feel, useful project storytelling, restrained motion, sound toggle as an optional interaction, and subtle 3D where it adds meaning.
- Performance, accessibility, responsive behavior, security, and clear project status should remain first-class requirements—not visual afterthoughts.

07 — Learning Journey & Technical Direction

Linux, networking and infrastructure

Linux is an active foundation area. The practical learning direction includes command-line confidence, file/process/service inspection, permissions, logs, environment variables, networking, and troubleshooting. This is the base layer for operating containers, CI runners, and Kubernetes nodes.

Cloud fundamentals

Through a Cloud Practitioner learning path, topics explored included on-premises versus cloud, IaaS/PaaS/SaaS, EC2, Elastic Beanstalk, Regions and Availability Zones, shared responsibility, and pricing models such as On-Demand and Reserved Instances. AWS labs and infrastructure work were previously explored, but AWS is currently deferred for the Notes App project; the current project target is local Kubernetes plus Docker Hub.

Current learning sequence (direction, not a guarantee)

STAGE	FOCUS	WHY IT MATTERS
1	Linux, shell, networking, Git	Understand the machine and the change history before automating it.
2	Docker and container fundamentals	Package software consistently and reason about runtime dependencies.
3	CI/CD and automated testing	Make verification repeatable and release feedback faster.
4	Kubernetes and GitOps	Operate workloads and reconcile desired state reliably.
5	DevSecOps controls	Add secret, dependency, code, image and runtime security checks.
6	Cloud deployment and reliability	Move from local labs to cloud infrastructure when ready and cost-checked.
7	AI-enabled products and automation	Build useful workflows with sound architecture, validation, privacy and human oversight.

Learning methods

- Concept-first learning: understand the purpose and trade-offs before running commands.
- Hands-on labs: validate ideas against real systems, logs, test outputs and reports.
- Active recall and spaced repetition: revisit concepts instead of relying on passive reading.
- Small-step troubleshooting: change one variable at a time and preserve evidence.
- Documentation: create GitHub-ready notes and explain the architecture, workflow, and lessons learned.

08 — Engineering Principles & System Design

Automation with feedback

Automate repeatable work, but make results visible through test reports, scan findings, logs, and clear failure signals.

Security by layers

Use multiple complementary controls: secret scanning, dependency audits, code-quality analysis, image scanning, secure HTTP headers, and runtime testing. No single tool proves an application secure.

Desired state and repeatability

Use versioned source and declarative deployment configuration where possible; prefer repeatable steps over undocumented manual changes.

Separation of concerns

Keep frontend, backend, data, deployment configuration, and security verification conceptually distinct so failures can be isolated.

Least privilege and secret hygiene

Limit workflow permissions and keep credentials out of source code and client-side bundles. Use appropriate secret stores and rotation practices.

Evidence-based claims

Tie a claim to an observed test, build, report, or manual check. Distinguish a passing pipeline from complete coverage or production readiness.

Human review for uncertain AI outputs

Surface disagreement and ambiguity; do not treat model confidence or consensus as ground truth.

Cost-aware progression

Use local labs when they answer the learning question; introduce billable cloud resources only when needed and after reviewing likely costs.

Security coverage map — Notes App

CONTROL	RISK IT HELPS ADDRESS	LIMITATION TO REMEMBER
Gitleaks	Accidentally committed secrets	Cannot find every secret in every context; prevent and rotate leaked credentials.
npm audit	Known dependency vulnerabilities	Only covers known advisories in the dependency tree; review severity and reachability.
SonarQube	Code-quality issues and selected static findings	Rules and analysis scope matter; a clean result is not a proof of correctness.
Trivy	Known vulnerabilities in container images and related artifacts	Findings depend on image contents, database freshness and configuration.
OWASP ZAP baseline	Passive web security observations and common configuration issues	Baseline is not a full penetration test; active scanning requires explicit scope and care.
Security headers / CSP	Some browser-side risks and unsafe resource loading patterns	Policy must fit application behavior; test functionality and monitor violations.

WHAT CAN BE HIGHLIGHTED

09 — Work Highlights & Evidence

HIGHLIGHT	WHAT IT DEMONSTRATES	EVIDENCE / CAVEAT
Built a local cloud-native application environment	Containers, Kubernetes workloads, Services, Ingress and service-to-service routing.	kind cluster and notes.local routing were recorded as working.
Moved from Jenkins to GitHub Actions	Modern CI workflow design and automated validation.	GitHub Actions is the current documented CI direction.
Added layered pipeline checks	Thinking about security and quality throughout delivery.	Gitleaks, npm audit, SonarQube and Trivy were integrated or run in the workflow.
Worked through a CSP finding	Interpreting DAST output, making a targeted configuration change and verifying behavior.	ZAP baseline reported missing CSP; enforcing CSP and other headers were added and manually checked.
Used Argo CD / GitOps	Understanding desired state and deployment reconciliation.	Application `notes-app` documented in Argo CD; verify live sync/health before a current claim.
Designed multi-model MCQ verification	AI product thinking, orchestration, validation and uncertainty handling.	Architecture/planning evidence; verify implementation status before claiming it is shipped.
Worked on Android collaboration product	Mobile UI, authentication, role-aware workflows and Firebase architecture.	Prior tests/builds and manual checks documented; some integrations remained pending.
Defined a distinctive portfolio direction	Product storytelling, design consistency and user experience awareness.	Design decisions recorded; verify current deployed implementation against the plan.

Suggested project demonstration pack

- Notes App: running application screenshot, GitHub Actions green pipeline, Trivy/ZAP report, Kubernetes workloads/Ingress, Argo CD application health.
- TeamSyncAI: screenshots of verified implemented flows plus a clear list of features still under development.
- AI MCQ extension: architecture diagram, user journey, provider comparison/uncertainty behavior, and a visible label for prototype versus planned features.
- Portfolio: live site and GitHub repositories, with short case studies explaining problem → architecture → decisions → security → results → next improvements.

WHERE I AM HEADING

10 — Roadmap & Future Vision

HORIZON	PRIORITY	SUCCESS SIGNAL
Near term	Finish and document the Notes App DevSecOps phases; confirm DAST behavior; keep local Kubernetes and Argo CD stable; continue manual verification before changing CI.	Reproducible pipeline, understandable reports, clear architecture notes and trustworthy screenshots.
Next	Continue policy enforcement learning with OPA/Kyverno after the current DAST phase; improve test depth and operational visibility.	Policies demonstrated against concrete cases; tests exercise application behavior, not just a smoke test.
After local foundation	Return to AWS/cloud deployment when the local workflow is stable and costs are understood.	Documented architecture, cost awareness, least privilege and cleanup plan.
Parallel product track	Advance AI MCQ consensus extension and explore social media agent workflows in bounded, testable increments.	Working prototype, provider abstraction, safe credential handling, human review and clear limitations.
Career	Build a strong evidence-based portfolio, publish concise project write-ups, and deepen core Linux/container/Kubernetes/cloud skills.	Projects explain the why, show proof, discuss trade-offs and identify what remains to improve.

Long-term vision

The long-term goal is not merely to collect tool names. It is to become the kind of engineer who can understand a system end to end, choose appropriate technologies, design clear boundaries, automate safely, investigate failures, communicate trade-offs, and deliver reliable outcomes. AI automation is an additional product-building direction—not a replacement for strong engineering fundamentals.

What I do not want to claim

- I do not claim to be a senior engineer or an expert in every technology listed.
- I do not claim the AI extension or social-media agent is fully production-ready unless current implementation evidence supports that.
- I do not describe a passing scan as proof that a system is completely secure.
- I do not describe local kind deployment as a production cloud deployment.
- I do not include AWS as part of the current Notes App target because it has been deliberately deferred.

A SIMPLE CONVERSATION GUIDE

11 — How to Explain My Work to Someone

30-second introduction

“I’m Nasir Hayat, a Computer Science student focused on DevOps and DevSecOps. I’m building practical experience by taking a web application through automated testing, security checks, Docker image scanning, Kubernetes deployment, Argo CD GitOps, and OWASP ZAP baseline testing. I also explore AI product ideas, including a browser extension that compares multiple AI answers to help surface agreement and uncertainty. My goal is to become a strong end-to-end engineer who can build, secure, automate, and operate real systems.”

Two-minute technical explanation

“My main lab is a Notes web app. Code is stored in GitHub, and GitHub Actions runs tests/build checks plus secret, dependency, code-quality and image scans. The frontend and backend are packaged as Docker images and deployed to a local kind Kubernetes cluster. NGINX Ingress exposes the application through notes.local, while Argo CD manages the GitOps deployment state. For runtime security learning, I use OWASP ZAP’s baseline scan against the running application. One example was investigating a missing Content Security Policy header, adding an enforcing policy and related security headers, then checking both response headers and application behavior. I’m treating this as an evolving learning project, so I separate verified results from future hardening work.”

Questions a technical reviewer may ask

- **Why GitOps?** It makes desired deployment state reviewable and lets a controller reconcile actual cluster state.
- **Why scan images after building?** The scanner needs the actual artifact being delivered; source dependency checks and image checks cover different layers.
- **What does ZAP baseline mean?** It is a passive baseline assessment, not a full active penetration test or proof of security.
- **What is the next security step?** Continue DAST verification, improve tests and observability, then learn policy-as-code with OPA/Kyverno.
- **What is the main learning outcome?** Understanding how code, tests, images, deployment configuration, runtime routing and security checks connect into one delivery system.

Quick glossary

TERM	PLAIN-LANGUAGE MEANING
CI	Continuous Integration: automatically validate code changes.
CD / GitOps	Delivery/deployment practices; GitOps uses version-controlled desired state reconciled by a controller.
Container image	A packaged filesystem and metadata used to run an application consistently.
Kubernetes Service	A stable network endpoint for reaching a set of workloads.
Ingress	A way to route incoming HTTP(S) traffic to services in a cluster.
SAST / code analysis	Analyzing source code for selected quality/security patterns without relying only on a running app.
DAST	Testing an application from the outside while it is running.
CSP	Content Security Policy: browser rules that constrain which resources a page may load or execute.
Consensus AI	Comparing multiple model outputs to reveal agreement or disagreement; consensus is not guaranteed truth.

THE DIRECTION

12 — Closing Profile

I am building my career through a combination of fundamentals, practical projects, security-aware delivery, and deliberate learning. My strongest current story is the Notes App DevSecOps lab because it connects source control, automated testing, security checks, containers, Kubernetes, GitOps, and DAST into a single system. TeamSyncAI and the AI MCQ verification extension show my broader interest in building complete products and exploring AI-supported workflows. My next step is to deepen reliability, testing, policy enforcement, documentation, and cloud skills while continuing to be precise about what is implemented versus planned.

BUILD WITH INTENT	SECURE BY DESIGN	LEARN WITH EVIDENCE
Solve a real problem and understand the architecture.	Layer controls and verify behavior instead of relying on assumptions.	Use tests, logs, scans and honest status labels to measure progress.

Prepared from information Nasir previously shared in ChatGPT conversations. Some project states may have changed since they were recorded. Before using this as a public résumé or recruiter document, verify current repository state, live deployment status, project links, and which features are implemented.